1.1.3 Contraintes sur les associations

a) Contrainte de partition

Une contrainte de partition permet de spécifier que les occurrences, d'une entité partagée avec une autre, ne peuvent participer à 2 associations. En d'autres mots, cela veut dire qu'une occurrence appartient obligatoirement à une association et exclut la participation à une autre.

Reprenons le cas de la réservation d'un vol, un client peut effectuer une réservation tout comme une agence de voyage. Dans les 2 cas il y aura une association entre l'entité demandeuse et l'entité réservation. En aucun cas une réservation ne pourra être effectuée par un client et une agence de voyage en même temps, cela devra se faire avec 2 occurrences différentes comprenant respectivement l'association adéquate.

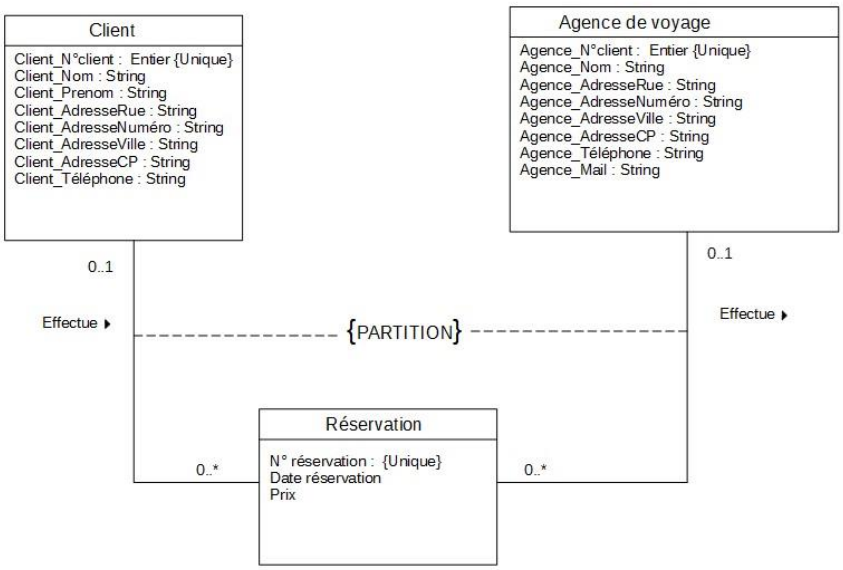


Fig. 22 : MCD - Contrainte de partition

La symbologie est exprimée en reliant les 2 associations sur lesquelles la contrainte est d'application par un trait pointillé avec le symbole {PARTITION}.

PARTITION = 1 ASSOCIATION OBLIGATOIRE

b) *Contraintes d'exclusion*

La contrainte d'exclusion est un cas particulier de la contrainte de partition, il s'agit bien d'exclure une association par rapport à une autre, mais dans ce cas, il n'est pas obligatoire de choisir une association définie dans le MCD.

Prenons par exemple le cas d'un paiement, un MCD peut contenir 2 associations permettant un paiement, soit un client effectue un paiement par carte de crédit, soit le client effectue un paiement par chèque.

La contrainte d'exclusion va spécifier qu'on ne peut à la fois effectuer le même paiement par chèque et par carte de crédit. Cependant, un client peut effectuer un paiement en argent comptant, ce dernier modèle de paiement n'est pas repris dans le MCD.

Le MCD ci-dessous exprime un modèle utilisant une contrainte d'exclusion.

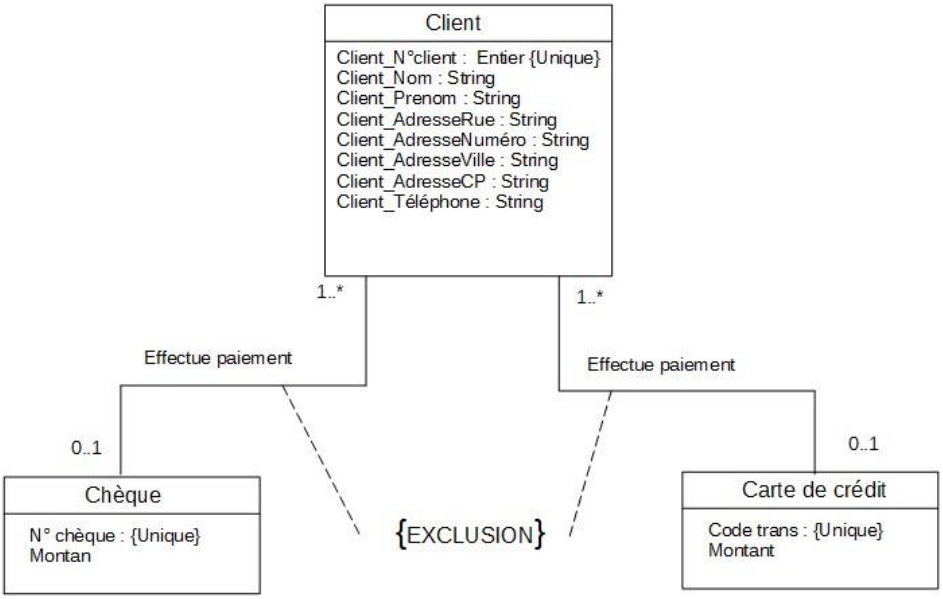


Fig. 23 : MCD - Contrainte d'exclusion

La symbologie est exprimée en reliant les 2 associations sur lesquelles la contrainte est d'application par un trait pointillé avec le symbole {EXCLUSION}.

EXCLUSION = 1 ASSOCIATION NON OBLIGATOIRE

c) *Contraintes d'inclusion*

La contrainte d'inclusion permet de spécifier qu'une association est aussi l'occurrence d'autres associations entre les mêmes entités.

Prenons l'exemple d'un employé d'une entreprise qui utilisent des logiciels intranet pour lesquels il doit avoir des accès. L'accès est une association entre l'entité employé et l'entité site ERP, l'employé doit effectuer une demande pour chaque accès. Lorsque l'employé se connecte sur un site avec un login, le système vérifie qu'il a bien accès au site demandé.

En réalité l'association de demande d'accès est une partie de la procédure de login et utilise les mêmes entités.

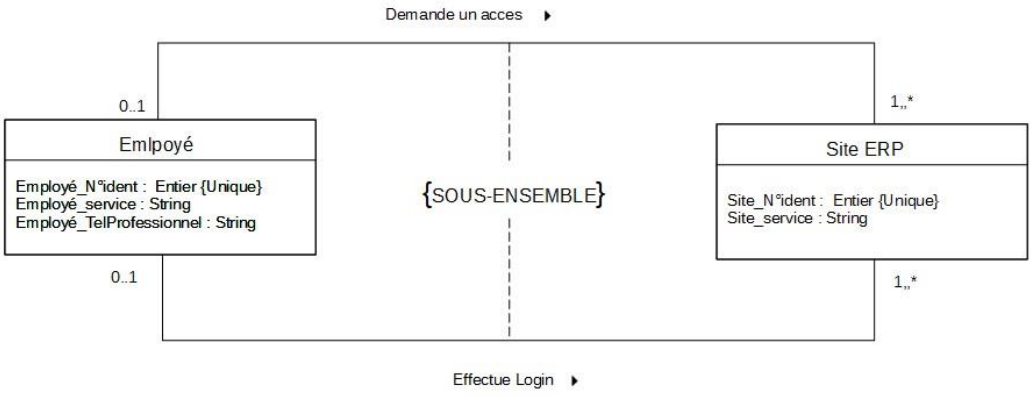


Fig. 24 : MCD - Contrainte d'inclusion

La symbologie est exprimée en reliant les 2 associations sur lesquelles la contrainte est d'application par un trait pointillé avec le symbole {SOUS-ENSEMBLE}.

INCLUSION = UNE ASSOCIATION EST UN SOUS_ENSEMBLE D'UNE AUTRE SUR LES MÊMES ENTITÉS

d) Contraintes de simultanéité

La contrainte de simultanéité placée entre deux associations reliant deux entités permet de spécifier qu'une occurrence sur une des associations entraîne automatiquement une occurrence sur la deuxième association.

Autrement dit, il y aura toujours une occurrence sur chaque association simultanée.

L'exemple ci-dessous montre que lorsqu'un client effectue un paiement (première occurrence), automatiquement un reçu est généré (deuxième occurrence).

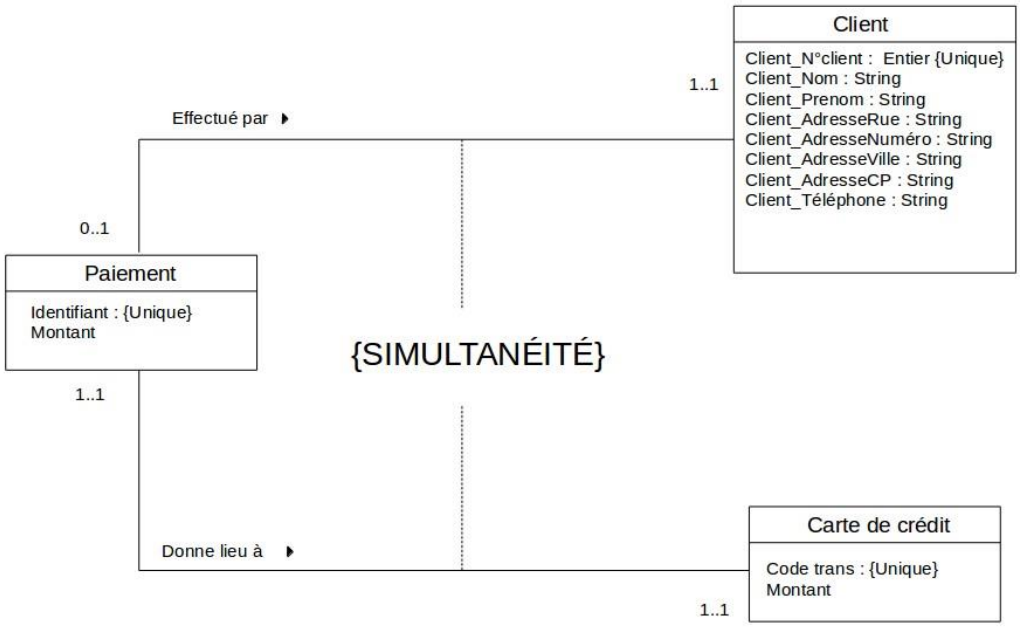


Fig. 25 : MCD - Contrainte de simultanéité

La symbologie est exprimée en reliant les 2 associations sur lesquelles la contrainte est d'application par un trait pointillé avec le symbole {SIMULTANÉITÉ}.

SIMULTANÉITÉ = OCCURRENCES SUR 2 ASSOCIATIONS EN MÊME TEMPS

1.1.4 Les notations

a) Notation de Chen

La notation de Chen utilise des rectangles pour formaliser des entités et des losanges pour formaliser les associations, il n'y a pas de notion d'entité d'association et si une association comporte des attributs, ces derniers sont simplement notés dans l'association correspondante.

On peut ne représenter que les multiplicités maximales sur les liaisons entre les associations et les entités, par exemple N au lieu de 1,N.

Tout comme dans la notation UML, la multiplicité du côté de l'entité d'arrivée indique le nombre d'occurrences de cette entité qui sont liées à l'entité de départ et seules les entités peuvent être reliées à des associations.

La notation de Chen n'autorise pas l'utilisation de contraintes sur les associations.

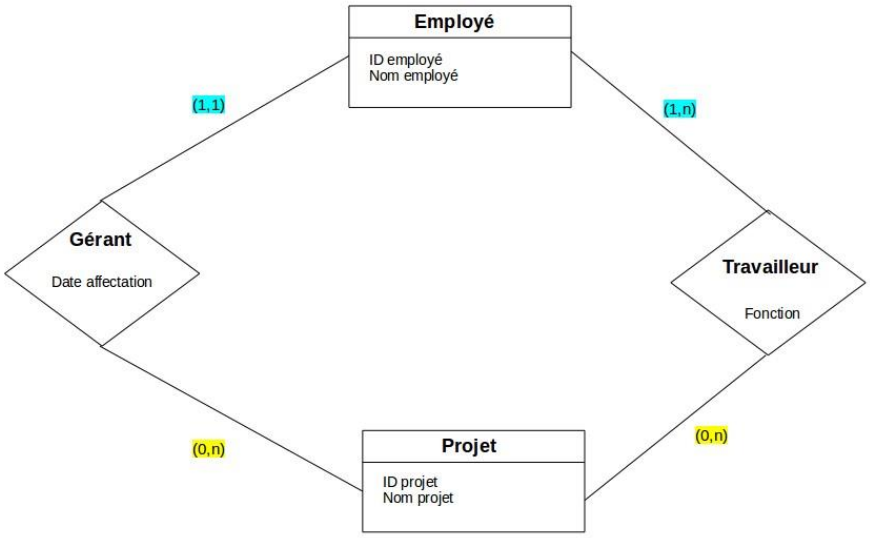


Fig. 26 : MCD - Notation de Chen

Le MCD ci-dessus peut être lu comme suit :

- ✓ Un employé peut être un travailleur sur aucun ou plusieurs projets
- ✓ Un employé peut être un gérant d'aucun ou plusieurs projets
- ✓ Un projet peut avoir comme gérant un seul employé
- ✓ Un projet peut avoir comme travailleur affecté un ou plusieurs employés

Tout comme dans la notation UML, les multiplicités peuvent être exprimées en ergot de coq. Il s'agit uniquement d'une notation différente qui n'altère en rien le formalisme.

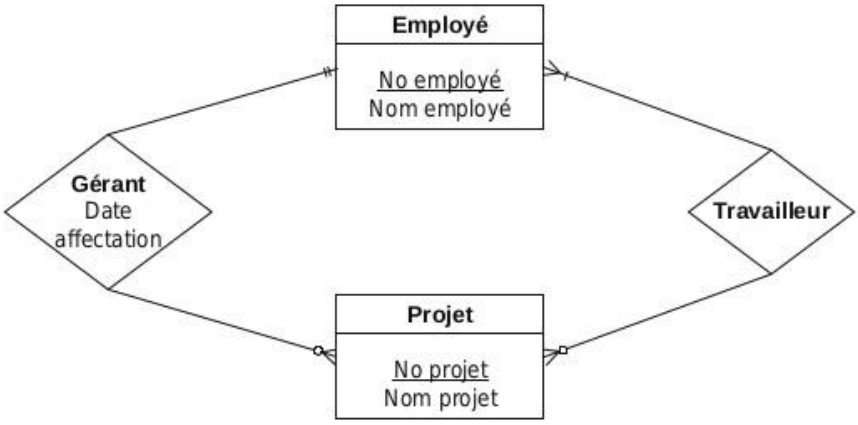


Fig. 27 : MCD - Notation de Chen en ergot de coq

Le tableau suivant reprend les codifications graphiques des multiplicités.

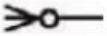
Multiplicité UML	Signification	Ergot de coq	Représentation numérique
0..1	Au plus un		(0,1)
1..1 (ou 1)	Un seul		(1,1)
0..* (ou *)	Un nombre indéterminé		(0,N)
1..*	Au moins un		(1,N)

Fig. 28 : MCD -Multiplicités

Il est possible que dans certains logiciels de modélisation de base de données, cette notation soit légèrement différente. Toutefois malgré ces différences, la symbologie reste très proche et intuitive tout en gardant les mêmes représentations.

b) *La notation UML*

En modélisation de base de données, la notation UML utilise la même sémantique que la notation de Chen, en effet cette dernière est en fait le prédécesseur de la notation UML.

Toutefois, elle permet plus de spécifications telles que les contraintes d'association. De plus, les association peuvent être considérées comme des entités.

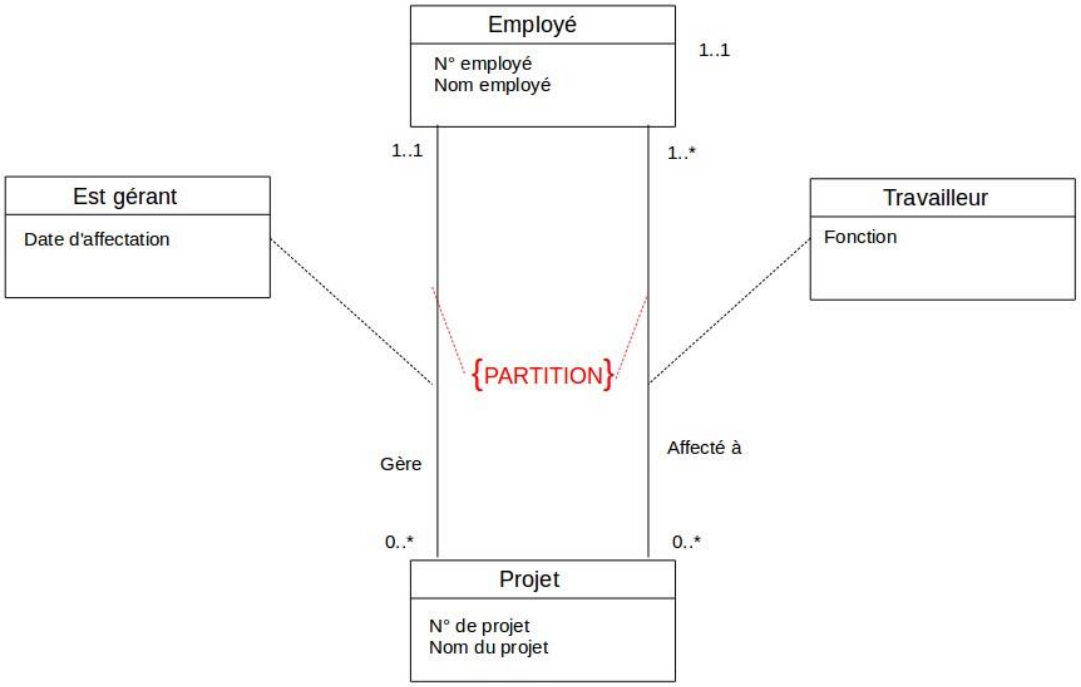


Fig. 29 : MCD - Notation UML

Dans le MCD ci-dessus, les associations du modèle en notation de Chen sont remplacées par des entités d'association, les multiplicités sont identiques et se lisent de la même manière.

Les fonctionnalités de la notation UML permettent de définir des contraintes d'association, dans cet exemple, nous avons défini qu'un gérant ne peut pas être un travailleur et vice-versa. Autrement dit, il ne pourra avoir une occurrence sur l'association "gère" ainsi qu'une occurrence sur l'association "affecté à" ayant chacune le même numéro d'employé.

Pour rappel, la contrainte d'association "PARTITION" utilisée dans l'exemple ci-dessus exprime qu'une occurrence se fait obligatoirement sur une des 2 associations proposées.

c) *La notation de Merise*

A l'instar de la notation UML, la notation de Merise est un ensemble de méthodes, de formalismes de notations et d'outils pour la modélisation de base de données. Il s'agit toujours d'un modèle entités-associations.

La grande différence entre Merise et UML, est la logique d'expression des multiplicités. Dans le cas d'une association binaire, la multiplicité située à côté de l'entité définit le nombre d'occurrences de l'entité d'arrivée qui participe à l'association. Ce qui représente une inversion complète des multiplicités sur les associations binaires par rapport à la notation UML ou de Chen.

Les association sont quant à elles représentées par des ovales.

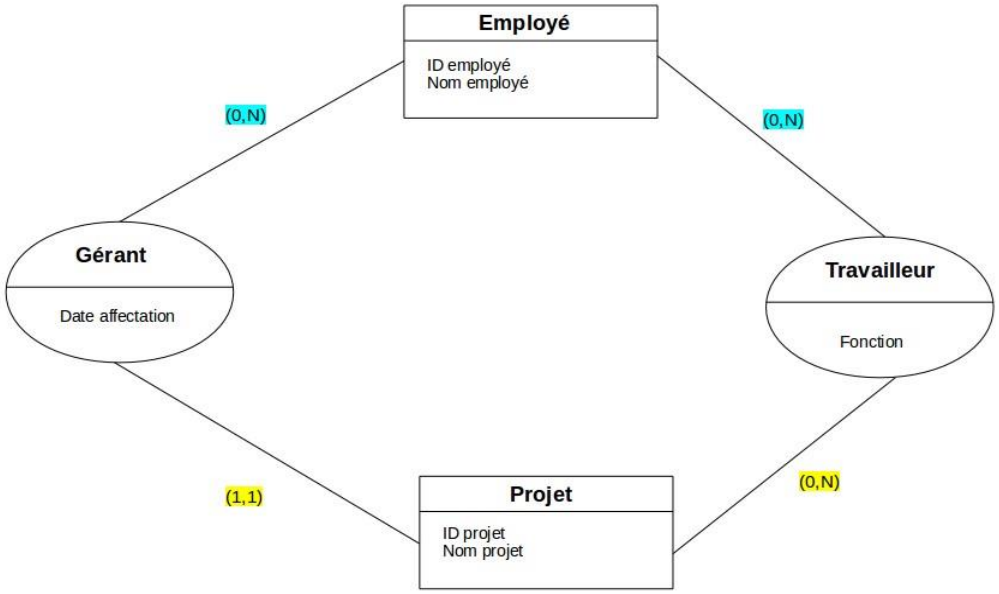


Fig. 30 : MCD - Notation de Merise

Le MCD ci-dessus peut être lu comme suit :

- ✓ Un employé peut être un travailleur sur aucun ou plusieurs projets
- ✓ Un employé peut être un gérant d'aucun ou plusieurs projets
- ✓ Un projet peut avoir comme gérant un seul employé
- ✓ Un projet peut avoir comme travailleur affecté un ou plusieurs employés

1.1.5 Les erreurs classiques

a) Le piège du ventilateur

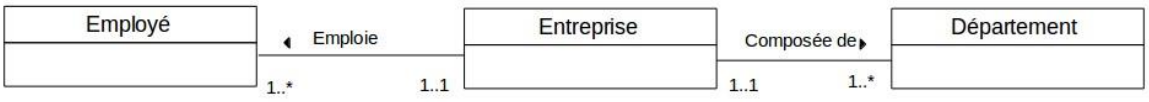


Fig. 31 : MCD - Ventilateur

Si dans l'exemple ci-dessus, le modélisateur souhaite définir à quel département appartient un employé, l'association "Emploie" va définir que l'employé travaille pour une entreprise et l'association "composée de " définit que l'entreprise possède un ou plusieurs département.

Dans le cas où l'entreprise ne possède qu'un département, il est facile de définir dans quel département l'employé travaille. En revanche si l'entreprise possède plusieurs département, il sera alors impossible de définir le département pour lequel l'employé travaille. Si on fait la lecture des associations en sens inverse, on s'aperçoit aussi que pour un département donné, on pourra avoir plusieurs employés.

Cela est dû au fait que dans la chaîne de lecture, on retrouve au moins une multiplicité maximale supérieure à 1, ce phénomène est appelé ventilateur. Le ventilateur empêche d'établir des relations entre les entités d'un modèle mal définit.

La solution au ventilateur est d'organiser les associations de manières à ce qu'une des 2 lectures ne comprenne aucune multiplicité maximale supérieure à 1.

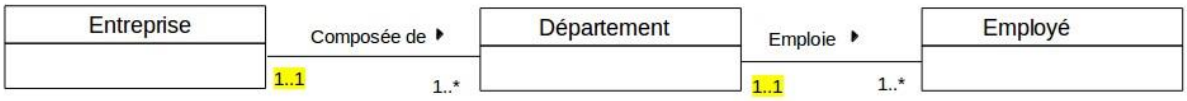


Fig. 32 : MCD - Solution au ventilateur

Dans le modèle corrigé, nous pouvons définir pour quel département de quelle entreprise un employé distinct travaille. En effet, dans le sens de lecture "Employé-Départements-Entreprise", l'association "emploie" présente une multiplicité maximale de 1 et l'association "composée de " présente également une association de multiplicité maximale de 1. Il n'y a donc plus d'ambiguïté possible sur plusieurs entités.

Le modèle ci-dessus présente en plus une hiérarchie cohérente des entités, une entreprise englobe un ou plusieurs départements qui eux-mes englobes un ou plusieurs employés.

b) *Confusion entre attribut et valeur d'attribut*

Certains cas de figure peuvent pousser le modélisateur à utiliser des attributs non appropriés.

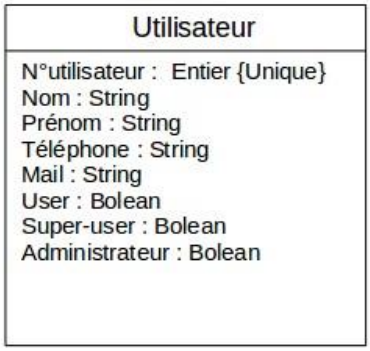


Fig. 35 : MCD - Confusion d'attributs

L'exemple ci-dessus illustre une mauvaise utilisation d'attributs, en effet, les attributs User, superUser et Administrateur sont en fait des valeurs d'attributs.

De plus, dans l'exemple cité, il sera nécessaire de définir des contraintes sur ces attributs, un utilisateur ayant des droits User ne peut pas avoir des droits Super-User et administrateur.

Le modélisateur doit veiller à utiliser les attributs adéquats afin de ne pas surcharger inutilement l'entité et éviter des contraintes inutiles.

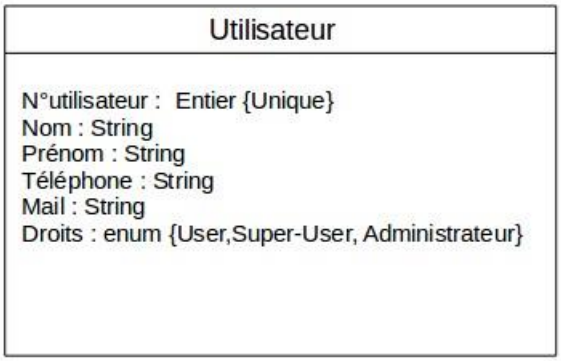


Fig. 36 : MCD - Solution pour la confusion d'attributs

L'entité ci-dessus répond de manière simplifié à la même demande que l'entité précédente.

1.1.6 Construction du modèle conceptuel de données

L'exemple de ce chapitre a pour objectif d'amener le lecteur sur une démarche réflexive permettant la construction d'un modèle conceptuel de données.

La méthode utilisée dans cet exemple didactique n'est aucunement exhaustive, mais donne un fil conducteur à la démarche de construction.

a) Établir la liste des attributs

Cette étude se fait avec la participation des utilisateurs du système d'information.

Le principe consiste à décomposer les besoins de l'organisation en un maximum de propositions élémentaires et à partir de ces propositions, extraire les attributs et les entités.

Les attributs peuvent être des données vitales exprimées par les utilisateurs ou recensées dans des documents relatifs aux fonctionnement de l'organisation.

Propositions :

1) Un client peut acheter plusieurs articles
2) Les achats effectués par un client sont effectués par une commande
3) Une commande est composée d'au moins un article
4) Un article peut être commandé plusieurs fois
5) Le montant de la commande est l'addition du prix total par article
6) Un client peut être enregistré sans avoir effectué une commandes

La proposition 1 nous permet de déduire aisément 2 acteurs qui seront des entités, une entité client et une entité article. Il s'agit maintenant d'en extraire les attributs.

Client	**Article**
Nom	Nom
Prénom	Prix
Age	Quantité
N° téléphone	
Email	
Adresse	
État civil	

La proposition 2 ajoute une entité commande qui est une transaction effectuée par un client. Les attributs d'une commande devront aussi être extraits. La proposition 3 stipule qu'une commande contient au moins un article, la proposition 5 indique que la commande contient un montant calculé.

Client	Article	Commande
Nom	Nom	Date
Prénom	Prix	Livraison
Age	Quantité	Article
N° téléphone		Montant
Email		
Adresse		
État civil		

Nous avons à ce stade lister les entités et les attributs qui découlent des propositions établies avec les représentants de l'organisation.

Nous pouvons constater que certaines entités ont des attributs avec le même nom, de plus certains attributs ne sont pas nécessairement vitaux aux fonctions de l'organisation, alors que d'autres peuvent être décomposés.

b) Épurer les attributs

- La règle de non-redondance interdit d'avoir dans un même modèle des attribut avec le même nom, c'est le cas pour l'attribut nom des entités client et article. Dans le cas présent, il s'agit bien de 2 attributs différents, une première solution serait de nommer les attributs avec un préfixe correspondant à l'entité, ce qui donnerait « client_nom » et « article_nom » ou bien changer le nom de l'article en « dénomination ».

- L'entité client possède des attributs qui ne sont peut-être pas vitaux pour l'organisation, nous pouvons supposer que les attributs âge, état civil peuvent être supprimés de la liste. Dans ce cas, il ne sera pas possible par exemple de faire des statistiques sur des commande à partir de l'âge et de l'état civil des clients. Si on laisse l'adresse Email, cela laisse supposer que les communications se font par téléphone ou par mail.

- L'adresse d'un client est un attribut complexe, en effet, une adresse contient une rue, un numéro, un localité et un code postal. Un code postal peut par exemple servir à estimer une distance pour calculer les frais de livraison, cette action serait rendue beaucoup plus complexe en traitement si cette information était dans un String contenant l'ensemble des coordonnées du client. Cet attribut peut être décomposé en plusieurs attributs indécomposable (atomiques).

Client	Article	Commande
Client_Nom	Article_Denomination	Commande_Date
Client_Prenom	Article_PrixUnitaire	Commande_DateLivraison
Client_AdresseRue	Article_Quantite	Commande_Article
Client_AdresseNumero		Commande_Montant
Client_AdresseVille		
Client_AdresseCodePostal		
Client_Email		
Client_Tel		

Le résultat de cette épuration donne le tableau ci-dessus, on peut constater que certains mots en Français ne comportent pas de caractères accentués, il ne s'agit pas là d'une erreur, mais plutôt d'une pratique qui évite des problèmes d'encodage par le SGBD. Les noms des attributs sont aussi choisis de manière à avoir une signification intuitive, « Client_AdresseNumero » stipule le numéro de l'adresse du client tandis que « Client_Numero » pourrait être un numéro d'identification d'un client.

c) Définir les identifiants et en déduire les classes

Dans le cas présent qui est relativement simple, les classes sont assez intuitives à définir. Dans le cas d'une étude sur un système plus complexe, le modélisateur doit veiller à ce que toutes les classes excepté les classes d'association, soient définies dans cette phase. Ce sera à partir de l'analyse métier, en définissant les différents acteurs et systèmes que le modélisateur arrivera à cette fin.

Il faut à présent définir les identifiants pour chacune de ces classes. Nous utiliserons à cette fin la règle d'identité.

Pour l'entité client, le nom ou le prénom ainsi que la combinaison des deux ne peut être un identifiant, en effet, plusieurs personnes peuvent avoir le même nom et le même prénom. La solution est donc de créer un identifiant unique comme un n° de client.

Il en est de même pour l'entité article, plusieurs articles peuvent effectivement avoir une dénomination identique, même s'il y a plusieurs variantes de ces articles, comme des fabricants différents. Il faudrait alors faire une combinaison des attributs dénomination et fabricant, solution ne représentant pas une bonne pratique. Si la dénomination comprend la référence du fabricant, cela pourrait dans cet exemple fonctionner, mais l'identifiant va dépendre de l'encodage de cet article, cette solution n'est pas stable car rien ne garantit que l'identifiant soit explicite. La solution sera aussi de créer un identifiant unique comme un n° d'article.

La classe commande est une classe abstraite, autrement dit, il n'y a physiquement rien qui puisse l'identifier de manière unique. Les attributs actuels ne permettent pas non plus d'identifier une occurrence de la classe. Dans ce cas, la solution sera encore la création d'un identifiant unique tel qu'un n° de commande.

d) Disposer les attributs dans les classes

Maintenant que les classes et les identifiants sont définis, il ne reste plus qu'à veiller à la disposition des attributs dans les classes correspondantes.

L'exemple étant simple, la disposition des attributs est pratiquement terminée. Cependant, il est intéressant de se pencher sur l'attribut « article_quantité ». La question à se poser est « que représente cet attribut ? ».

S'il représente la quantité d'articles en stock pour cette occurrence, sa place est bien dans la classe article, il serait alors plus intuitif de renommer cet attribut de manière plus explicite, comme « article_StockQty ».

En revanche, s'il représente la quantité d'articles commandés pour une occurrence, sa place devrait être dans la classe commande. Il faut alors le renommer « Commande_quantité ».

L'attribut « Commande_Article » correspond à l'article commandé, cet entité est identifiée par l'attribut « Article_N°Article », cette redondance ne peut exister (règle de construction).

La répartition des attributs devient la suivante :

Client	Article	Commande
Client_N°Client {unique}	Article_N°Article {unique}	Commande_N°Commande {unique}
Client_Nom	Article_Denomination	Commande_Date
Client_Prenom	Article_PrixUnitaire	Commande_DateLivraison
Client_AdresseRue	Article_Stock	Commande_Montant
Client_AdresseNumero		Commande_Quantité
Client_AdresseVille		
Client_AdresseCodePostal		
Client_Mail		
Client_Tel		

Il ne reste plus qu'à définir les domaines des attributs (string, entier, …).

A ce stade, chaque classe est identifiée correctement par la règle d'identité.

Les attributs sont non redondants (règle de non-redondance), unique (règle de description) et dépendent bien de l'identifiant de la classe correspondante.

Enfin, aucun attribut d'une entité se trouve dans une autre (règle de construction).

e) *Déterminer les associations entre les classes*

Les propositions 1 et 2 précisent qu'un client peut acheter un ou plusieurs articles en effectuant une ou plusieurs commandes. La proposition 6 quant à elle précise qu'un client peut être enregistré sans qu'il n'ait encore effectué une commande.
Ceci nous amène à définir une association entre la classe Client et la classe Commande.
L'association ne concerne qu'un seul client, la multiplicité sera donc 1..1, comme un client peut effectuer plusieurs commande et qu'il peut être enregistré sans qu'il n'ait effectué un première commande, la multiplicité du côté de l'entité commande sera 0..*.
Cette association se lit « un client n'effectue aucune ou plusieurs commandes ».

L'association entre la classe Commande et la classe Article est complétée par les propositions 3 et 4. Ceci nous amène à définir pour cette association une multiplicité de 1..* du côté de l'entité Article, car une commande contient au moins un article. Du côté de l'entité Commande, on peut déduire qu'un article peut ne pas être commandé et un article peut aussi être commandé plusieurs fois donc plusieurs commandes, la multiplicité sera de 0..*.
Cette association se lit « une commande peut comporter un ou plusieurs articles et un article fait l'objet de plusieurs ou d'aucune commande ».

Le modèle conceptuel sera alors :



Fig. 39 : MCD - Exemple de définition associations (phase1)

Cependant, notre analyse est encore incomplète, nous constatons que l'association entre la classe commande et la classe Article est d'une multiplicité N..N. Contrairement à l'association « Effectue » pour laquelle il est facile de déterminer quel client effectue une telle commande, car une commande n'est que pour un seul client.

Dans le modèle actuel, l'association « Comporte » peut relier plusieurs articles à plusieurs commandes et la quantité d'articles est définie dans la classe commande, ce qui veut dire que chaque quantité d'un article commandée dépendra d'un numéro de commande différent. Cela ne répond pas à la proposition 3, la quantité d'article commandée ne peut donc pas se trouver dans la classe Commande.

La question à se poser est comment définir une quantité d'articles différents pour plusieurs commandes distinctes ?

Le moyen de modéliser cette question réside dans l'utilisation des classes d'association ou classes faibles. Cette classe a la particularité de relier l'association binaire aux identifiants des classes reliées. Cette classes appelée Article commandé associera un numéro de commande et un numéro d'article. Cette classe n'a donc pas besoin d'identifiant et peut contenir des attributs propre à cette association.

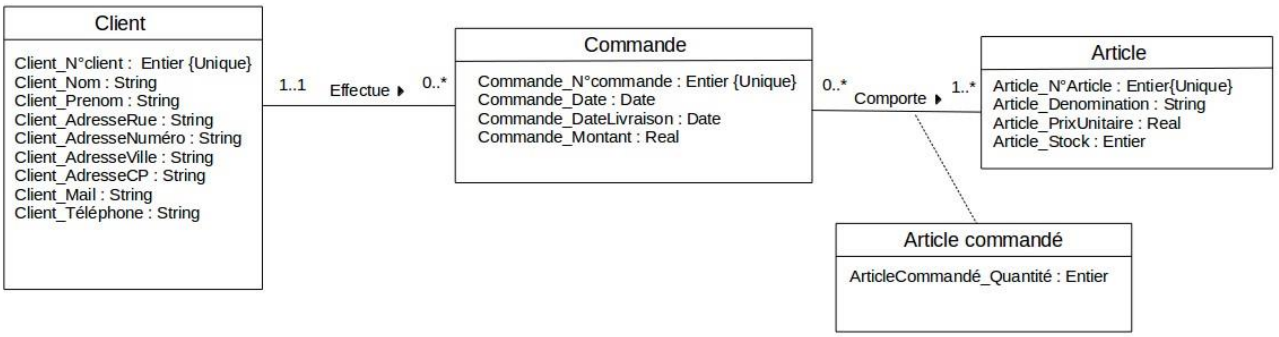


Fig. 40 : MCD - Exemple de définition d'associations (phase2)

Cette entité faible peut aussi contenir des informations propres au moment de la création de l'association. Dans notre exemple, supposons que le prix unitaire d'un article augmente entre la commande et la livraison, le client devra se voir facturé le prix au moment de la commande, or le prix unitaire de l'article dans la base de données aura été modifié. La solution est de placer un attribut « ArticleCommandé_PrixUnitaire » dans la classe d'association. Les règles sont respectées car il ne s'agit pas d'une redondance, en effet, cet attribut ne représente plus le même prix unitaire bien que les valeurs peuvent être identiques.

Le modèle devient alors :

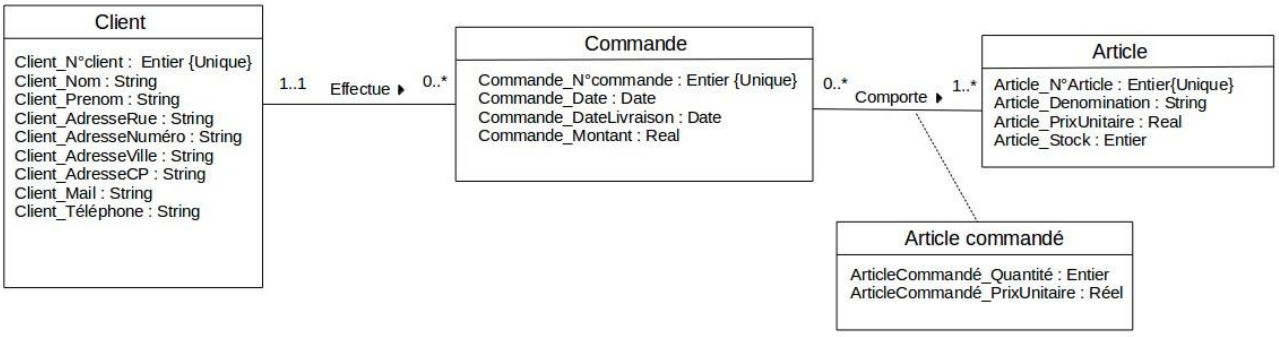


Fig. 41 : MCD - Exemple de définition d'associations (phase3)

Cette pratique permettra d'effectuer les calculs de montant en tenant compte des prix attribuées aux différents articles lors la création de la commande.

f) Mise en place des contraintes

Il ne reste plus maintenant qu'à placer les contraintes sur notre modèle.

Dans cet exemple simple, nous avons déjà définit une contrainte sur les identifiants, ces derniers sont en effet définis comme uniques.

D'autres contraintes peuvent être ajoutées, par exemple, lorsque l'utilisateur entre des données, certains champs peuvent être vide et d'autres sont obligatoires. Pour ces derniers nous pouvons ajouter la contrainte sur les attributs non nul.

Dans l'exemple présent, nous pourrions définir que dans la classe client, au moins l'attribut mail ou l'attribut téléphone doivent remplis, en d'autres mots un de ces 2 attributs peut être nul. La manière de modéliser cette contrainte est de placer un commentaire avec une explication naturelle

Le modèle conceptuel de données final sera :

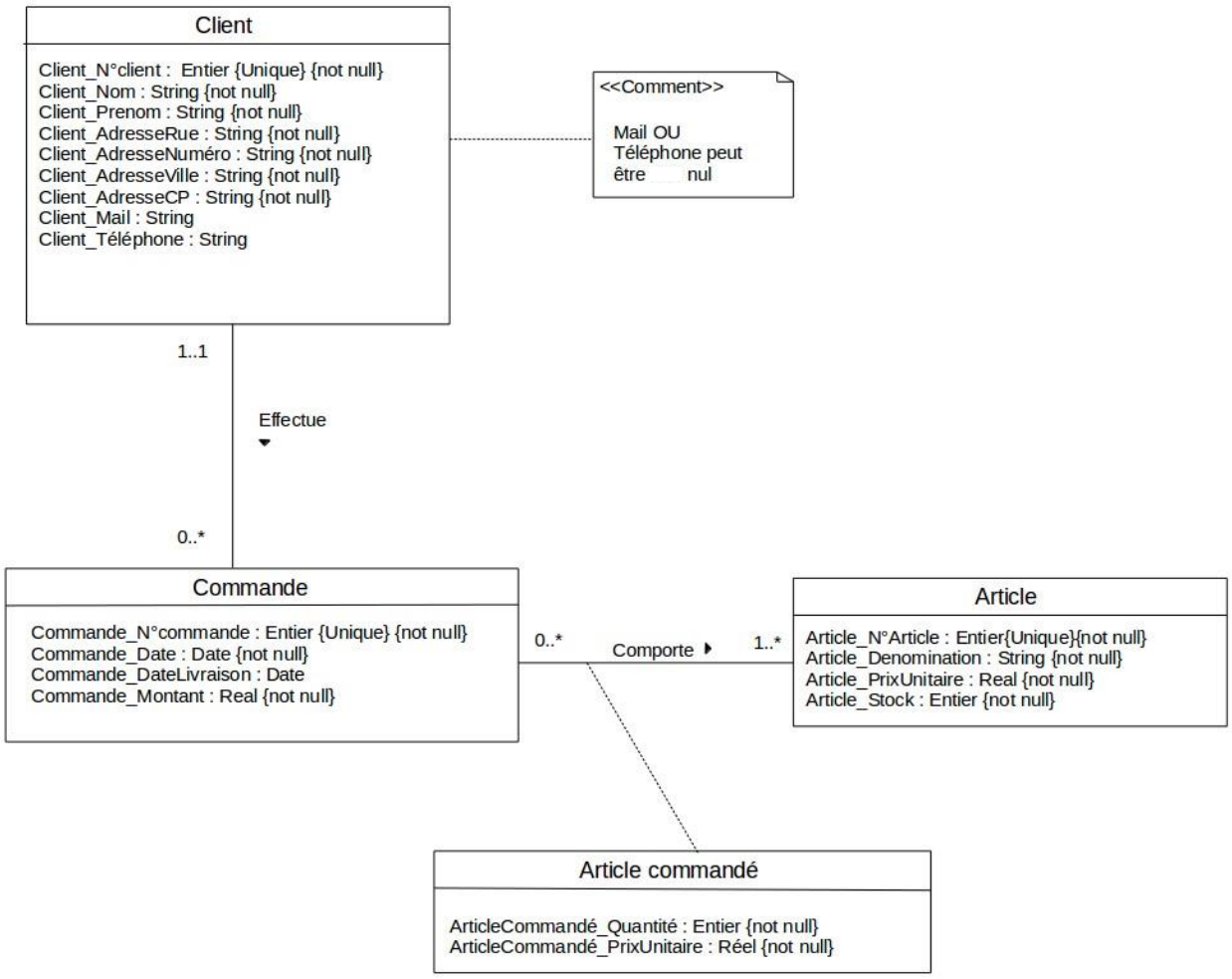


Fig. 42 : MCD - Exemple de construction finale